

Brushless Carwash

The Three

Problem Statement:

The goal of this project is to successfully transform .pdc data of a car into an accurate 3D mesh which will then be exported as an STL file, ensuring important details such as mirrors, tow bars and bull bars are properly identified and included.

This will be achieved by a series of steps in a standardized pipeline that will receive the raw .pdc data, removing any detail that is not in reference to the car. We consider the “noise” in following:

- Floor
- Ceiling
- Side walls
- Below curtain
- Statistical outlier (Points that are significantly distant from neighbors)
- Isolated point clusters (Small groups of points disconnected from the main point body)

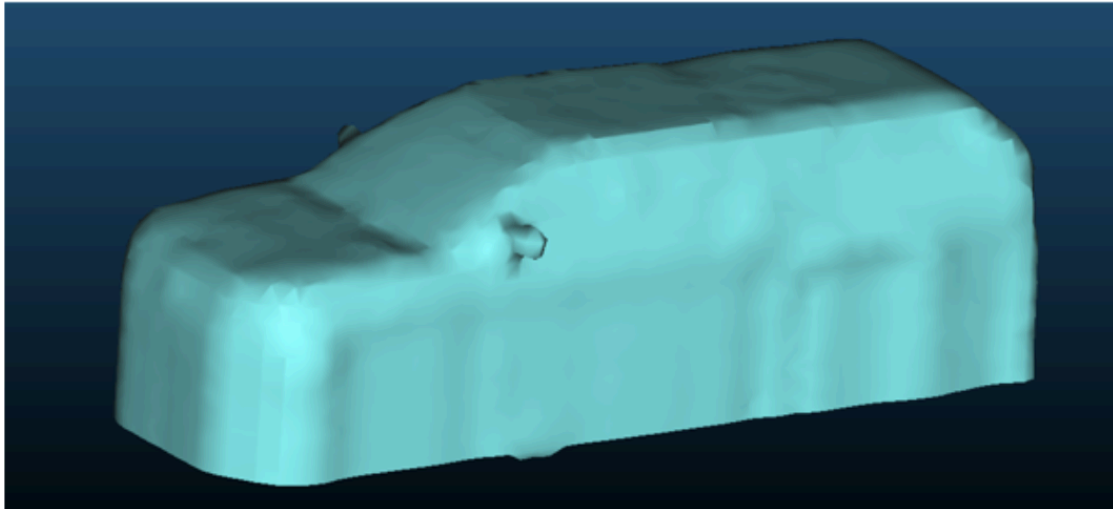
We highlight important parts of the car such as:

- Wheels
- Mirrors
- Roof of the car
- Front extreme (bull bar)
- Rear extreme (tow bar)

Which then will be transformed into a mesh that will be used to safely wash the car automatically.

The end POC: Working program/converting software

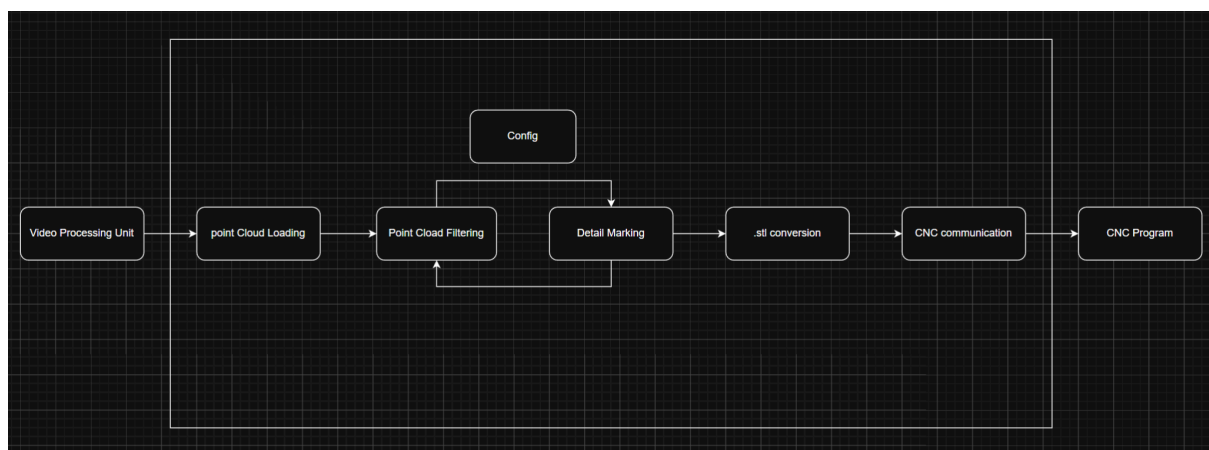
Success criteria: We aim to reach results like pictured here, but more precisely and as detailed as possible (detailed mirrors, bull bar ...)



Work plan:

1. Extract raw point cloud data(in .pcd) from the OVP-based camera system
2. Filter noise and erroneous measurements
3. Detect and preserve critical vehicle features
4. Identify and handle abnormal or unsafe data conditions
5. Convert the processed point cloud into a watertight, high-fidelity 3D mesh
6. Export this mesh as a single STL file suitable for downstream path-planning algorithms

The proposed flow:



Research techstack:

- Language

- Libraries
- Extensions
- Applications to work with

Week 1 – Project Progress & Discoveries

In the first week of the project, our main focus was understanding how to work with 3D point cloud data. This was completely new for us, so a lot of time was spent exploring how point clouds are structured and how they can be processed using Python and Open3D.

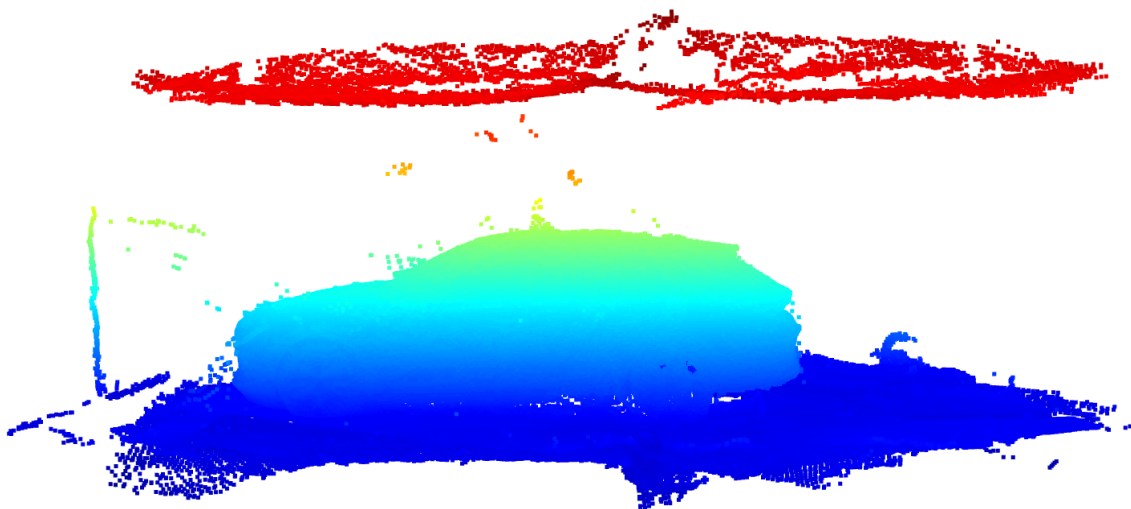
We learned how to load and visualize point cloud data and quickly discovered that raw data contains a lot of noise and unwanted elements, such as the floor and ceiling. Initial attempts to remove the floor using simple height thresholds were not very reliable. This led us to research more advanced techniques like plane segmentation using RANSAC.

Through experimentation, we discovered that detecting planes works well, but removing them correctly is more challenging than expected. We learned that downsampling improves stability for plane detection, and that checking the plane's orientation (using normals) is important to distinguish the floor from walls or other surfaces.

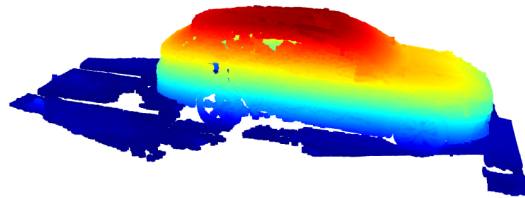
Overall, this week was mainly about exploration and learning. While the solution is not yet fully robust, we now have a much better understanding of point cloud processing and the challenges involved. This forms a solid foundation for improving accuracy and robustness in the coming weeks.

Example 1:

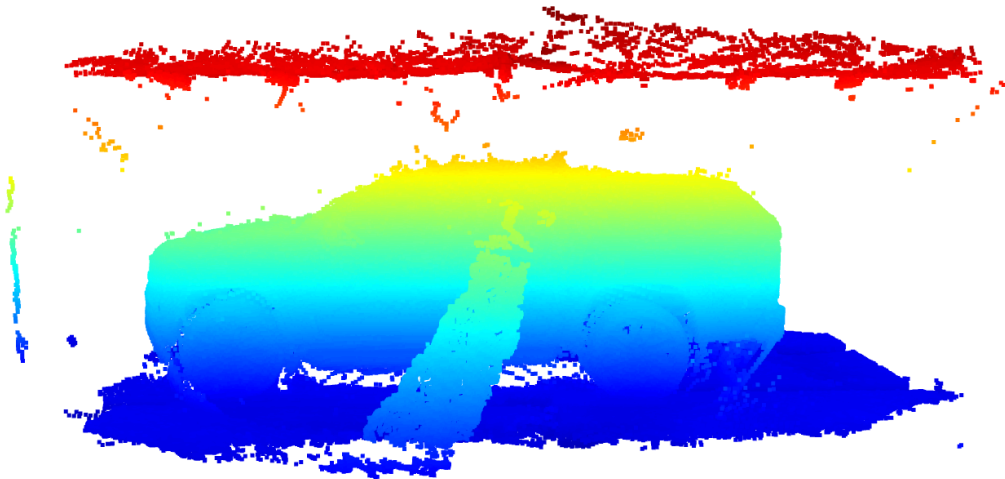
Original Point Cloud Data file:



Detected and removed ceiling, yet the floor is partially removed.

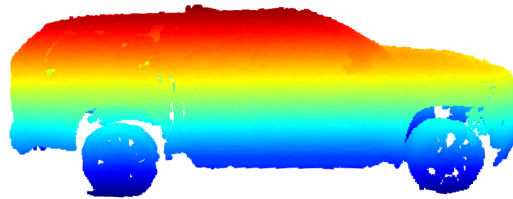


Example 2: Amarok met schuim
Original Point Data Cloud:



Main Car cluster after detecting floor and ceiling, and filter them. In this example it worked

better.



Week 2 :

The goal of this week was to understand the structure and quality of raw point cloud data coming from the OVP-based camera system, and to make the concept of “noise” and “invalid measurements” concrete and visible.

This week we focused on exploring a single raw .pcd file of a car with foam applied. Since this data was new to us, the main focus was not on producing a final mesh yet, but on **understanding what is actually inside the point cloud**.

We implemented a first-stage diagnostic script that:

- Loads and inspects the raw point cloud
- Analyzes the coordinate system and bounding box
- Detects large planar surfaces such as the floor
- Categorizes every point into meaningful groups
- Identifies important car features
- Produces both visual and textual diagnostic output

Findings and discoveries

1. Raw data contains a lot of non-car data

From ~922,000 points, only ~42% belongs to the actual car. More than half of the data consists of floor reflections, side walls, noise, and measurement artifacts. This clearly shows why raw data cannot be used directly.

2. “Noise” and “invalid measurements” are different

We learned to distinguish:

- **Invalid measurements** (floor, walls, below-curtain points) caused by the environment or setup
- **Noise** (statistical outliers and isolated clusters) caused by sensor instability, reflections, or foam behavior

By visualizing these categories with different colors, the difference becomes clear for both developers and customers.

3. Floor detection is critical

The floor alone accounted for more than 50% of all points. Detecting the lowest horizontal plane proved to be a reliable way to identify and classify floor data.

4. Important car features are detectable early

Even before mesh generation, we were able to detect and highlight:

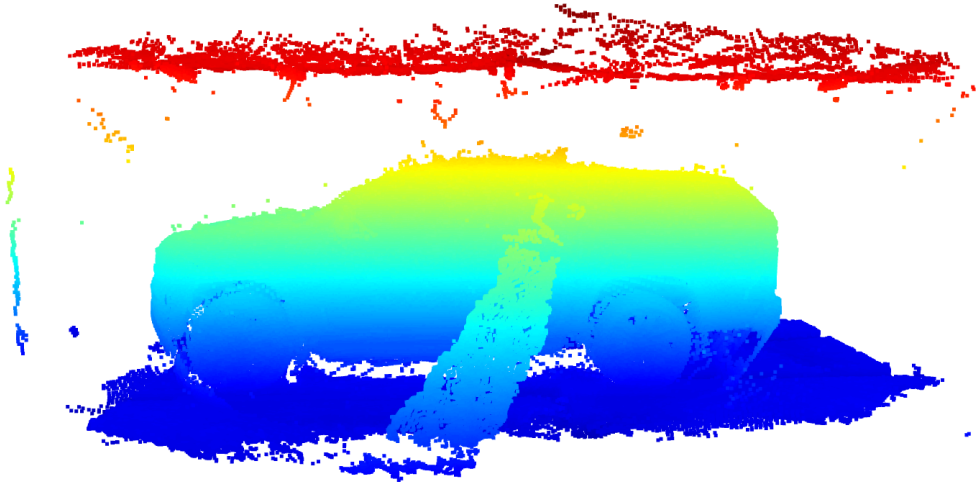
- Wheels
- Mirrors
- Roof area
- Front and rear extremes (possible bull bar and tow bar)

This confirms that feature preservation must already be considered in early filtering stages.

5. Dimensions become reliable after cleaning

After removing noise and invalid data, the measured car dimensions were realistic and within configured limits. This validates the cleaning approach.

Original point data cloud:



The program detects:

Total points: 922,624

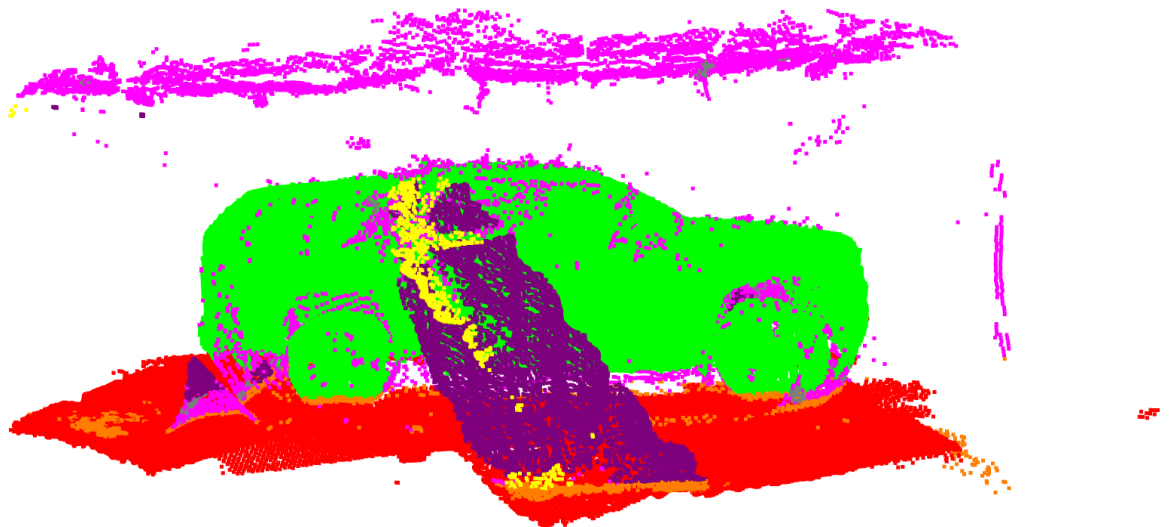
Bounding box:

X: -0.331 to 8.667 m (range: 8.998 m)

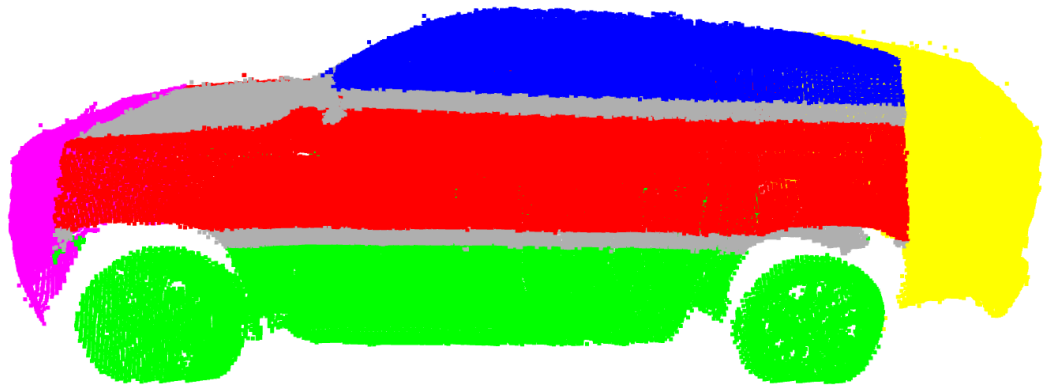
Y: -0.107 to 5.105 m (range: 5.212 m)

Z: -0.381 to 3.025 m (range: 3.406 m)

- **Data quality analysis - color coded :**



Car features highlighted:



- Green = Wheels
- Red = Mirrors
- Blue = Roof area
- Yellow = Rear extreme (tow bar)
- Magenta = Front extreme (bull bar)

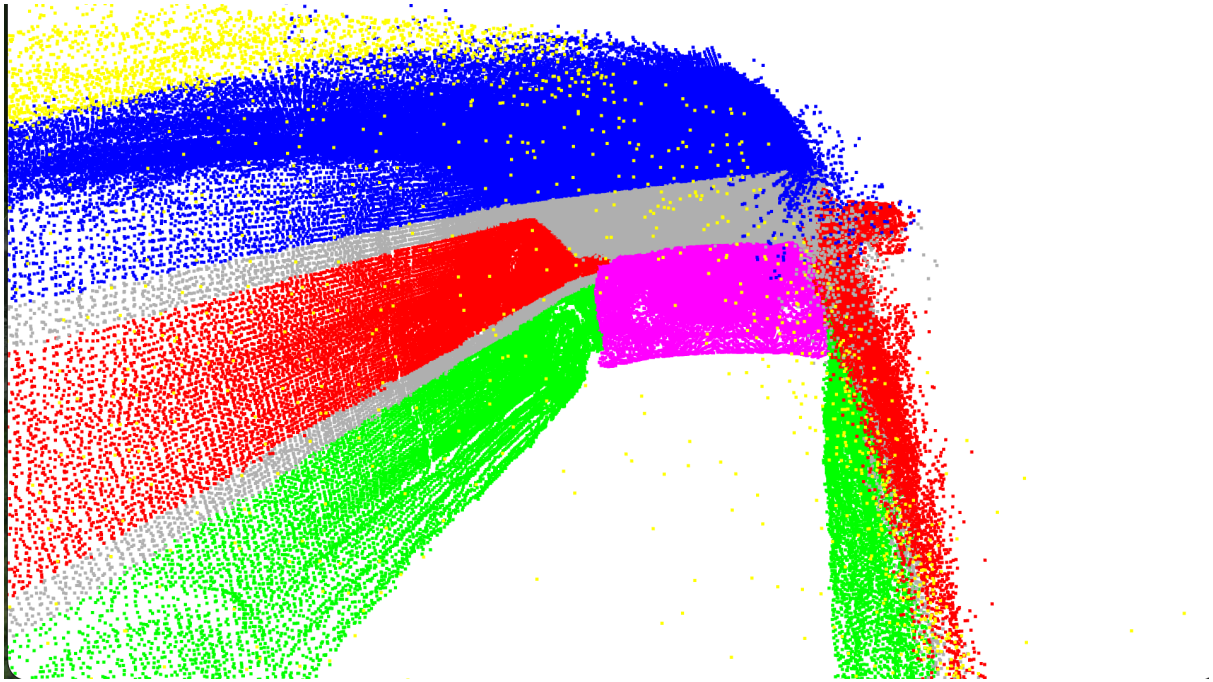
Week 3-4:

We've created a config.yaml file where customer can tune the parameters to change the configuration of filtering mechanism, here is an example:

We have these variables by default, based on results of filtering we can tune to make filtering more aggressive and efficient in removal of noise points(have to know the balance tho, because if these parameters are too high, the filtering becomes too aggressive)

With these parameters, the filtering worked not so well, so we have to tune:

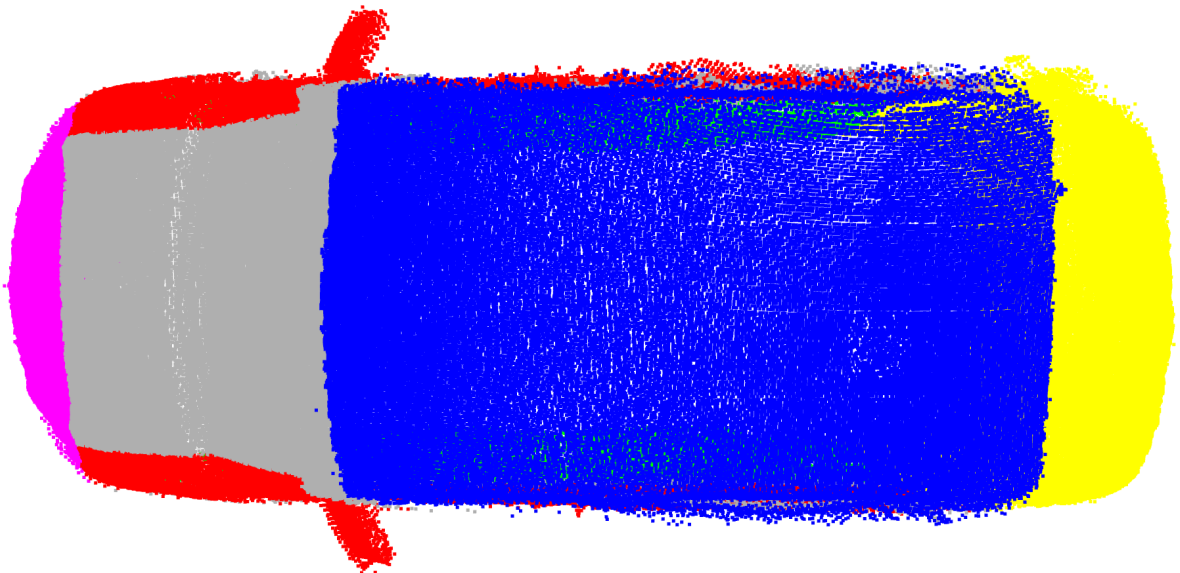
```
noise_detection:
  # Statistical outlier removal
  outlier_neighbors: 100      # Number of neighbors to analyze (recommended: 15-30)
  outlier_std_ratio: 1.3     # Standard deviation threshold (recommended: 1.5-3.0)
```



As we can see we have these extra points on the side of the car, they weren't filtered, so we can tune the parameters mentioned above to reach a better result:

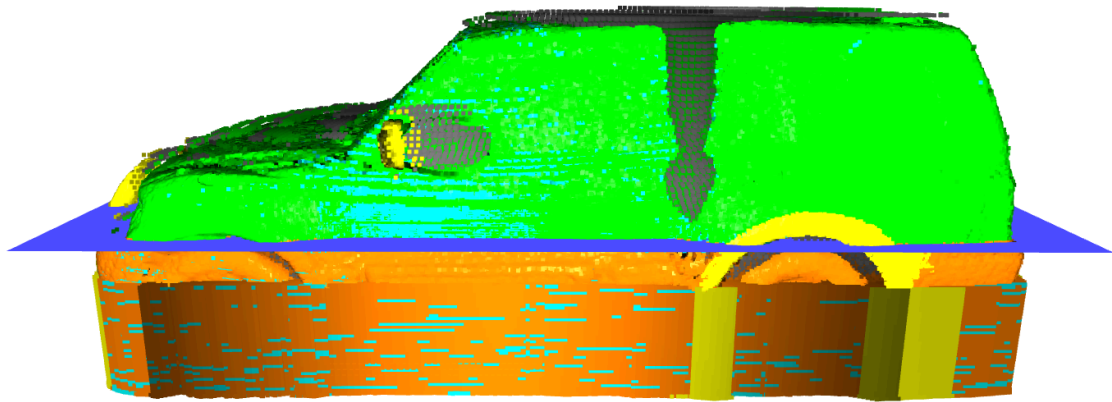
```
noise_detection:  
  # Statistical outlier removal  
  outlier_neighbors: 140      # Number of neighbors to analyze (recommended: 15-30)  
  outlier_std_ratio: 0.9     # Standard deviation threshold (recommended: 1.5-3.0)
```

And as you can see, on the right side of the car there are much less noisy points.



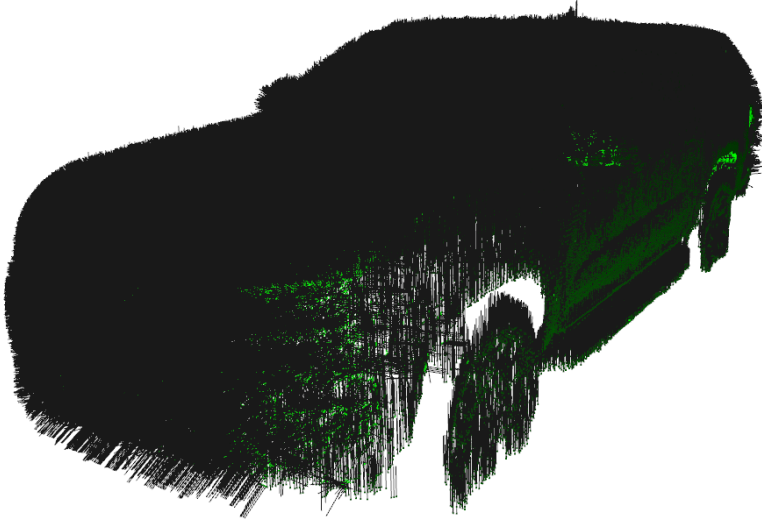
The client asked for the curtain to be from the top part of the wheel down to the floor.

Firstly, we had to detect the highest Z point of the wheel area, note the height in the report and draw a plane there to put the curtain down from that point:



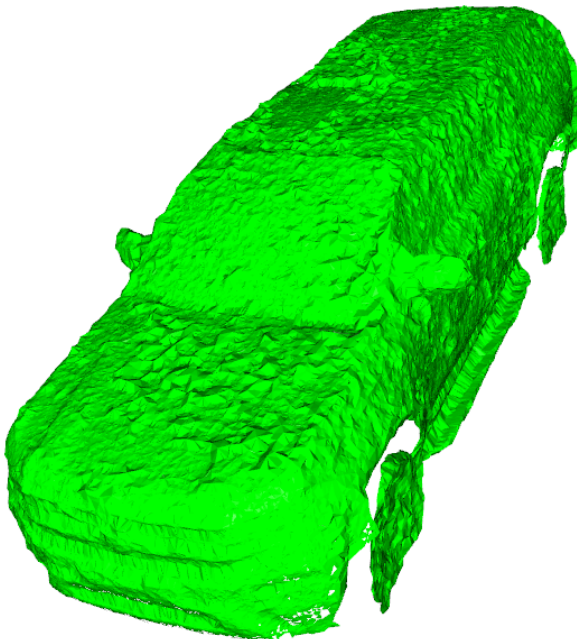
After the first week i got to this result

After using rufina's cleaned cars I started looking into meshing algorithms. Apparently we needed to add normals to the points in order to create meshes. This wasn't all too difficult. (black lines)

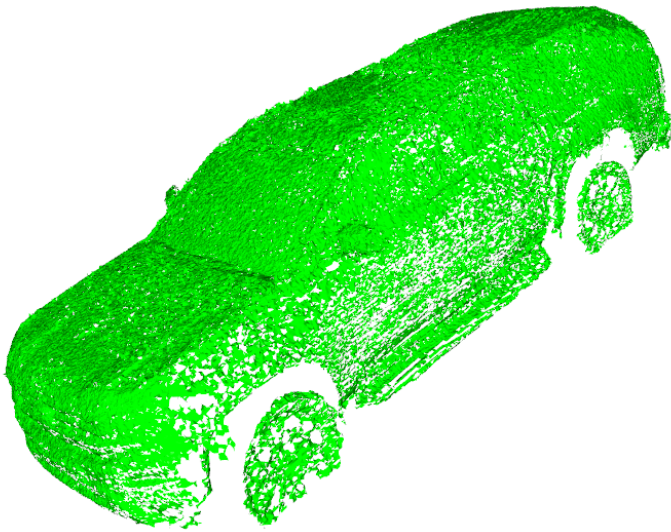


So now time to try out some meshing algorithms:

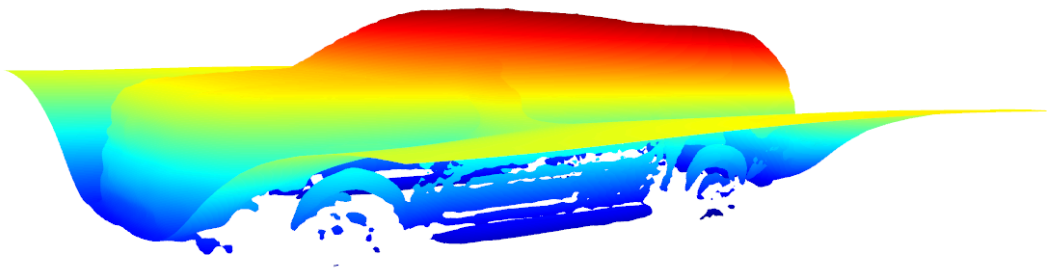
Alpha:



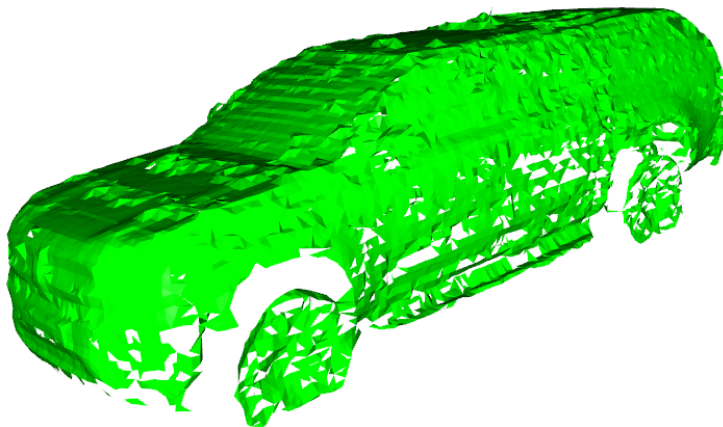
Ball Pivoting:



Poisson:

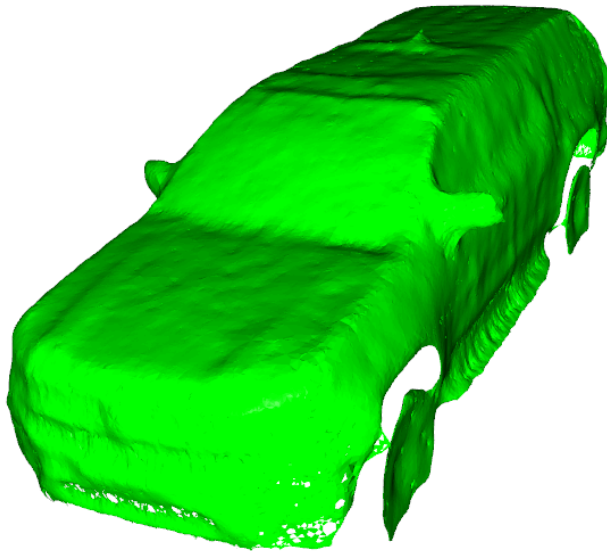


After this I continued experimenting a bit with the ball pivoting since it was better at catching details than alpha but never got it to be nice without holes, even after downsampling

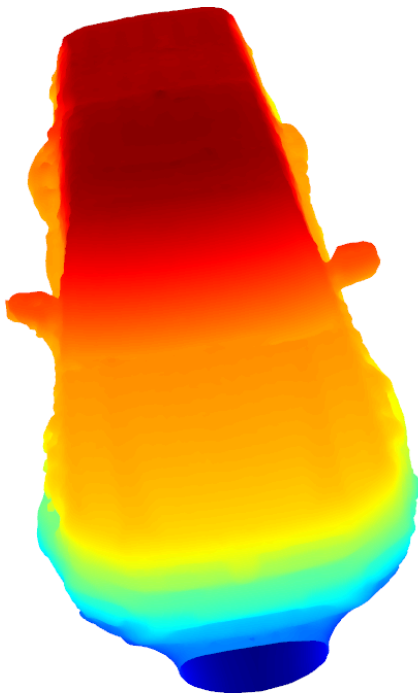


So I decided to continue with alpha reconstruction for now.

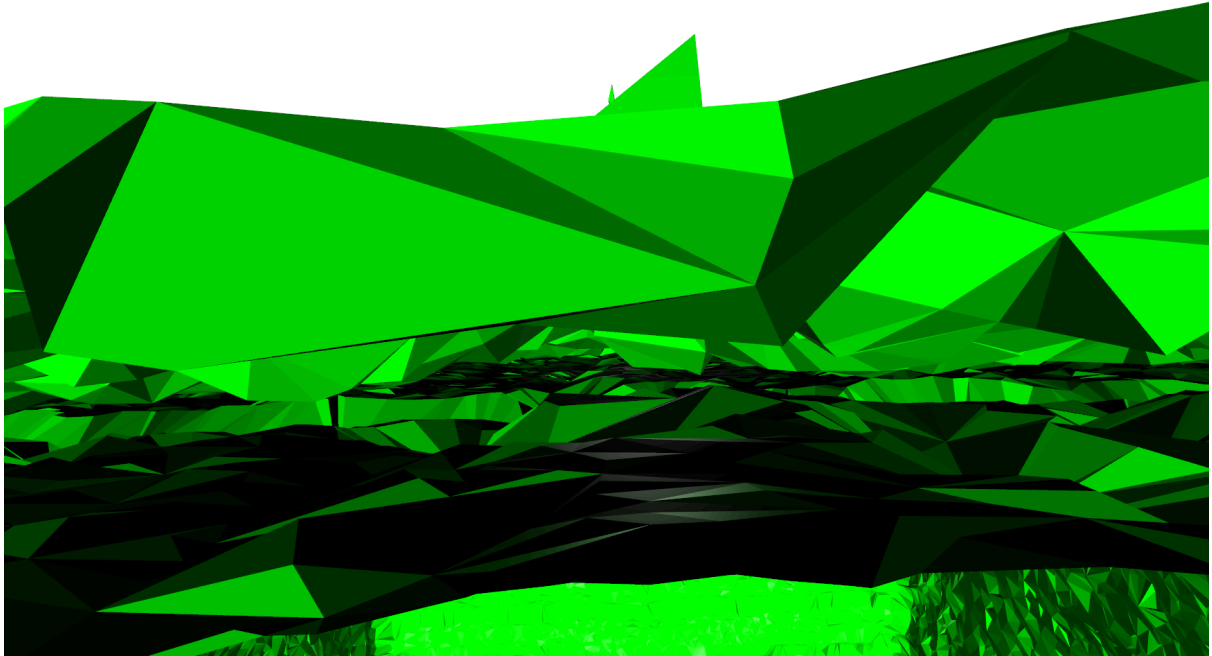
Seeing as it looked a bit rough I decided to apply some smoothing with smoothing filters. Averaging filter would make the car ever so slightly smaller but gives a smoother look then laplacian filter so i decided to to use the averaging filter.



All of the sudden I noticed the normals were a bit weird, if we look back we can see that on the sides the normals are pointing upwards instead of outwards, after making them point outwards i got this result with poisson so I started trying out both a bit:



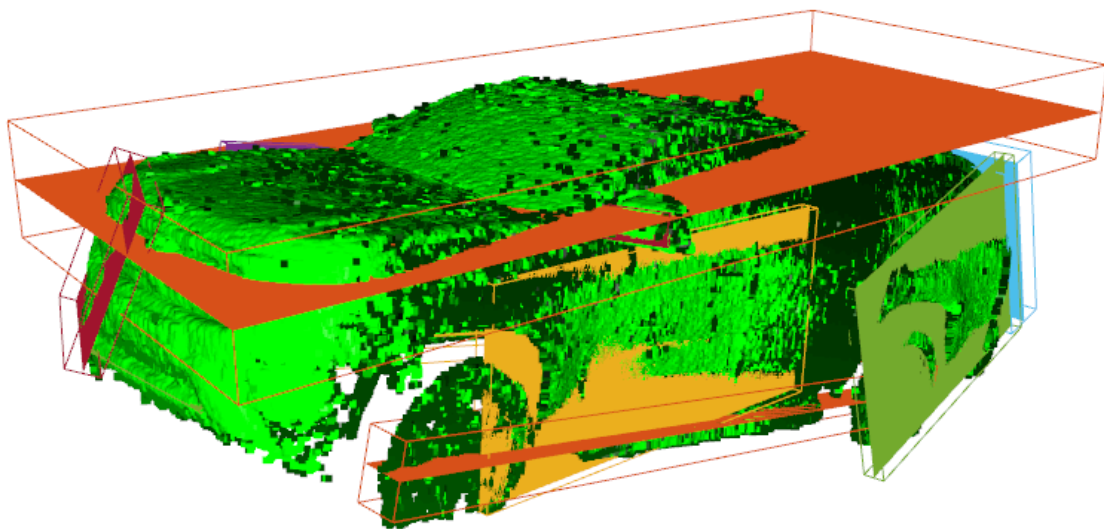
After looking closer at the alpha shapes (and a bit at the ball pivoting) I noticed that we were creating a sort of double shell where our shell was not just 1 layer but actually 2 layers making it a full object. If we zoom into the roof we can see the following:



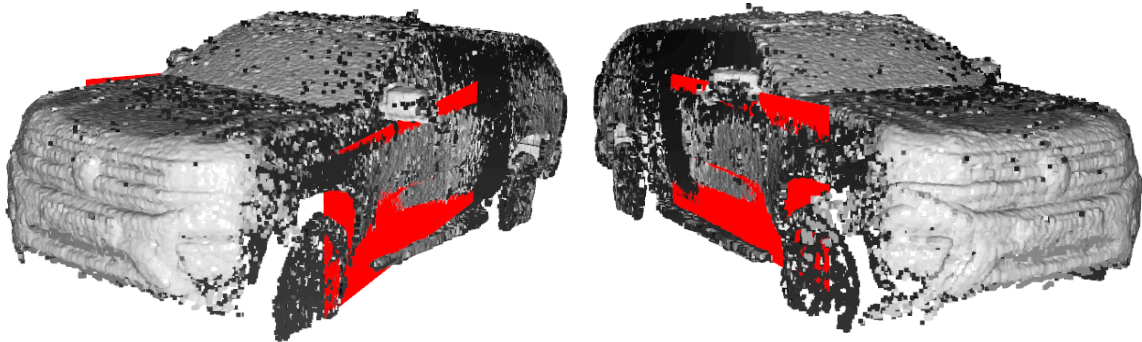
After discovering this and noticing that poisson does not have this problem I decided to only continue with poisson from now on.

Now I wanted to try and get rid of the weird bulge on the sides of the car so started experimenting with cutting off the sides and cleaning them more separately.

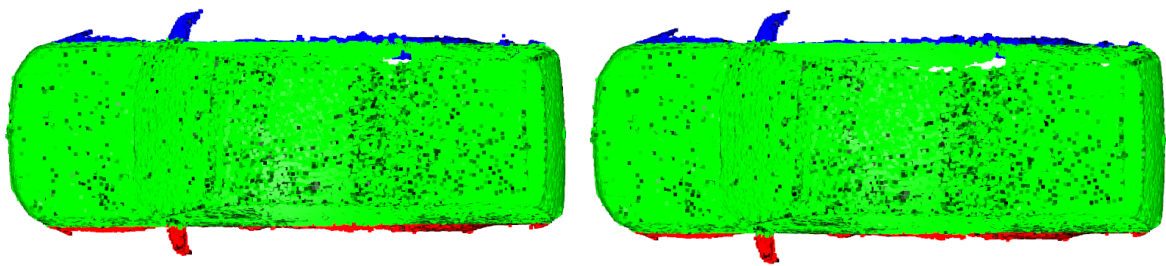
To do this I wanted to experiment with finding the side plane of the car so I could take it off, clean it, and put it back on. Here is a result i got for cutting of the sides of the car:



After filtering for the 2 planes with the biggest y component (sideways) I ended up with these 2 planes:

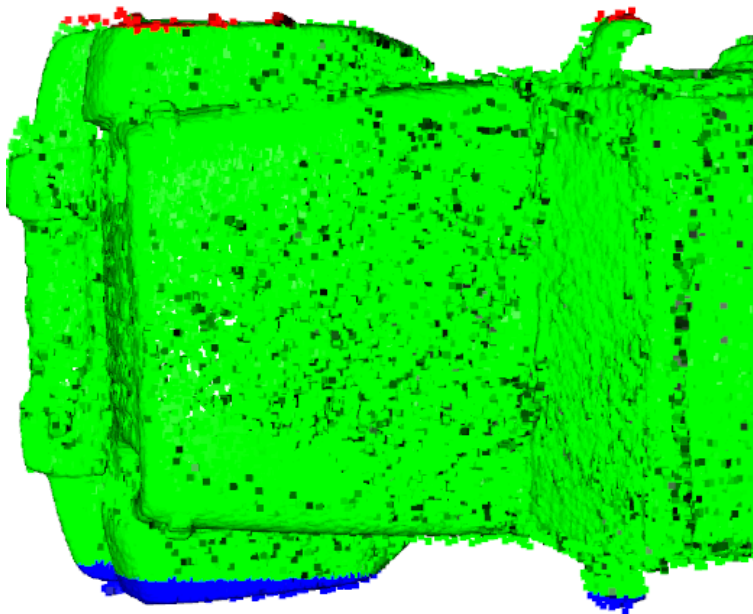


After separating the sides and cleaning them we get this result (with some painting to indicate sides):



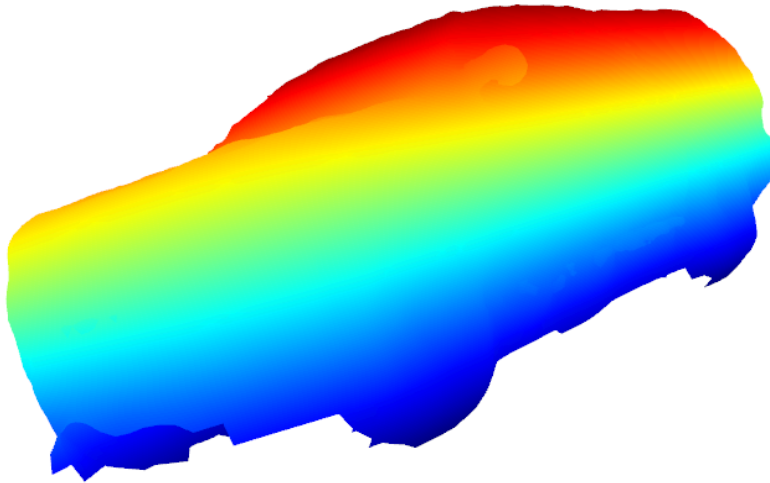
And after reconstruction the bulge is still there, just a bit smaller.

After running this on a different car (which is unlikely to pass by often) i decided to not use it anymore since it didn't even capture the mirrors or doors (which i wanted to capture):

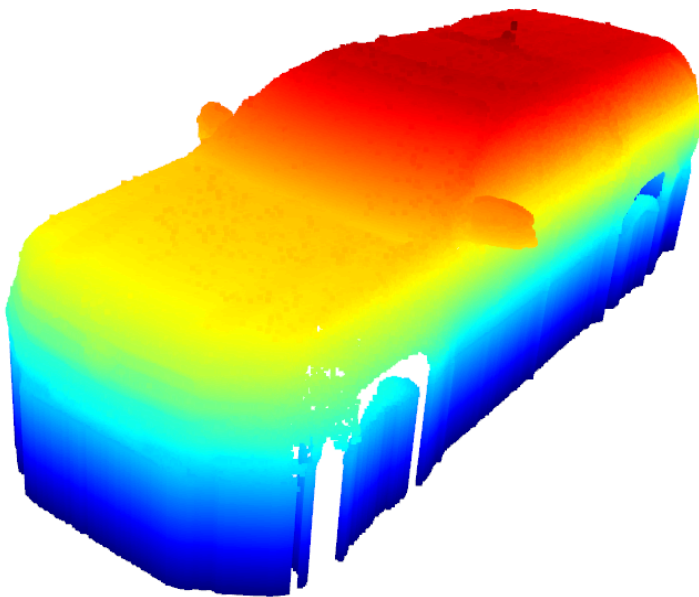


After some more looking into cleaning and trying out meshlib instead (paid library) I noticed that when using the meshlib normal estimation the bulge I had before wasn't there anymore so I started looking into different ways to estimate the normals with open3d. After a bit of work I got a similar result without the bulge.

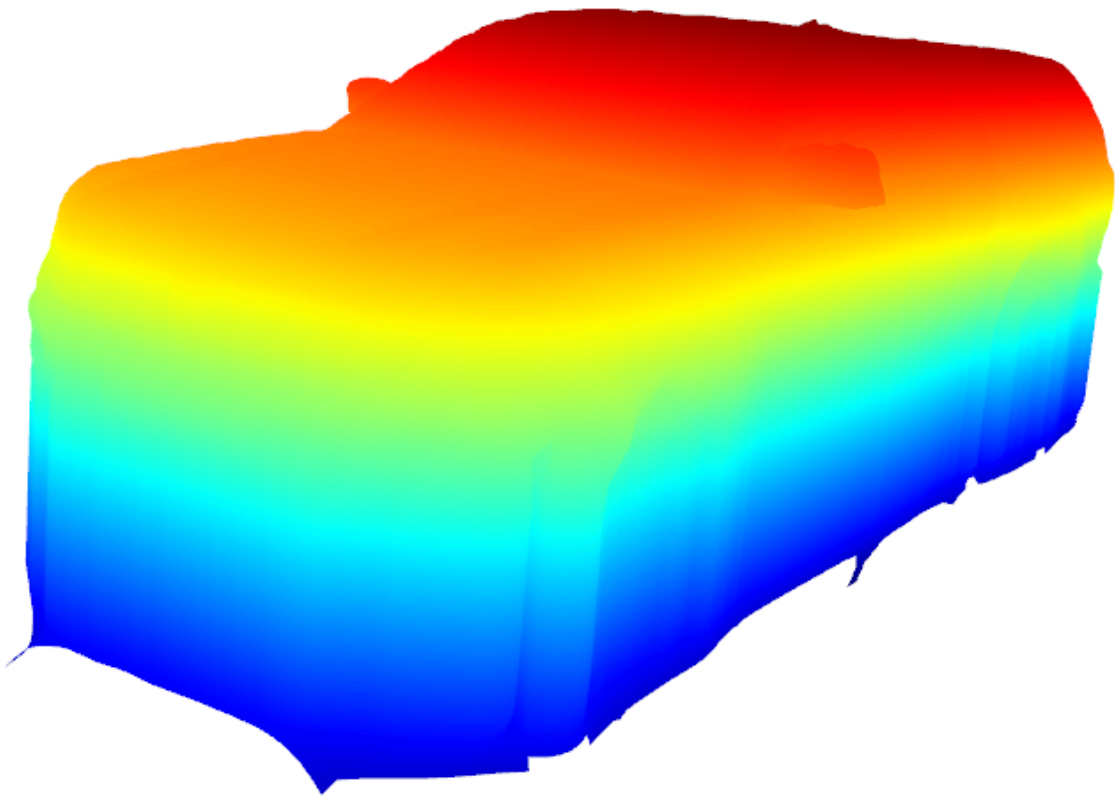
Now I get rid of the curve poisson adds to close the shape to have an open bottom:



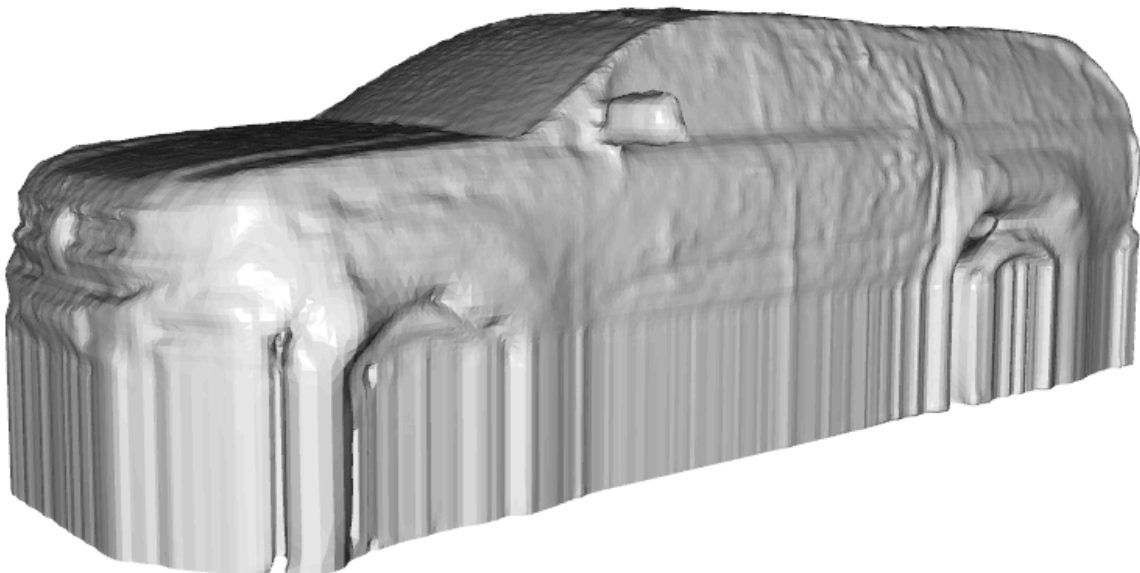
Now I started looking into making the sides drop down to 0 so we have a nice “curtain” going down to the floor.



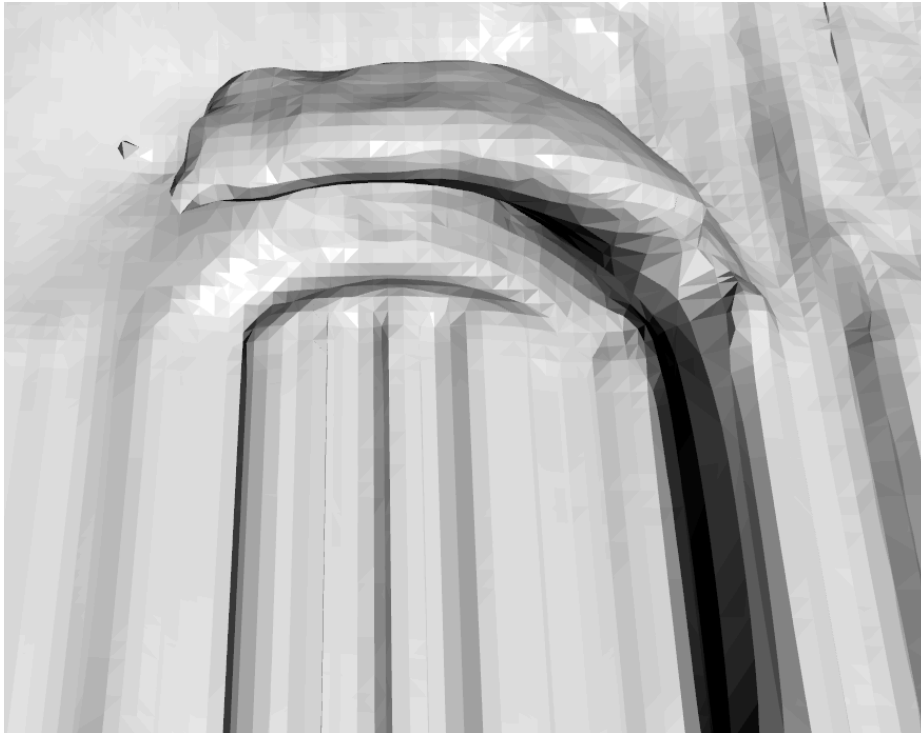
Using the reconstruction and opening the bottom we get the following result:



This however is not a nice flat bottom and not all at $z=0$ so I looked into cutting it instead of deleting “uncertain” parts. At first I ended up with a jagged edge but after changing to cutting the triangles (and replacing them with smaller ones) instead of removing all below $z=0$ I got this result:



This might look good on the outside and especially the top part but we still got a lot of weird stuff happening in the wheel wells:



To combat this I started looking into getting rid of anything on the inside of the car. I ended up using a method where i place 8 cameras on the outside of the car and if a point is detected by 1 of the cameras (not covered by another point with a certain radius around it) and the weird inside artifacts were pretty much gone (picture is taken from the bottom):



Using AI Foundation Models to Recognize Car Parts in Point Clouds

1. Problem Context

Our current approach uses:

- Geometric heuristics (bounding box positions)
- Height thresholds
- Clustering
- Statistical filtering

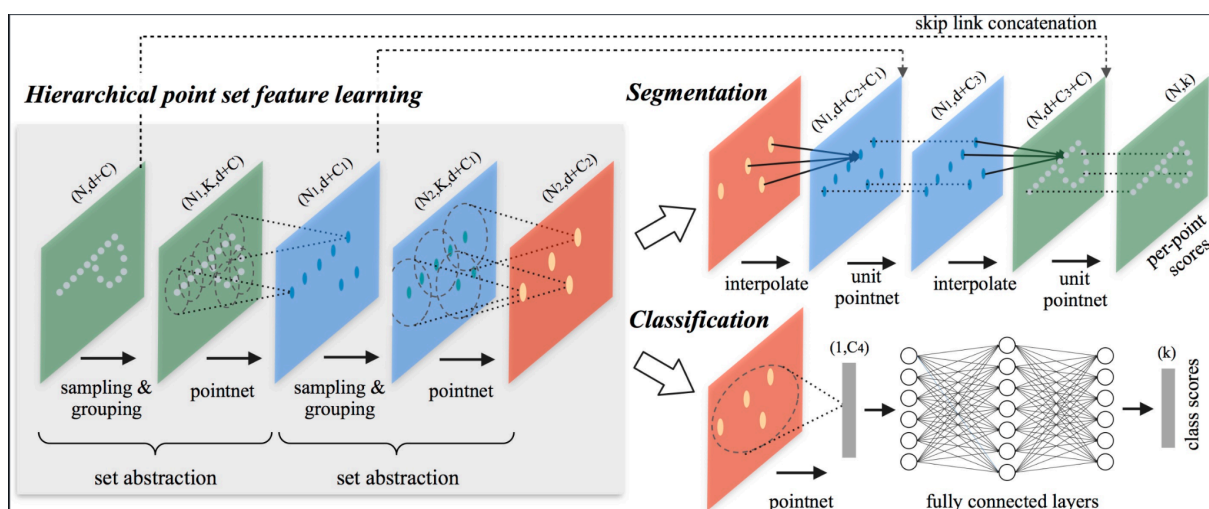
This comes with certain benefits such as time, it's pretty fast, explainable and it works well with little data. But also it has some disadvantages: hard to generalize, breaks on unusual cars and needs manual tuning.

AI recognition could help improve robustness. There are 3 realistic categories that could be used given that we have a time constraint, we will focus on PointNet and PointNet++..

2. Pretrained 3D Deep Learning Models

PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space

Created by Charles R. Qi, Li (Eric) Yi, Hao Su, Leonidas J. Guibas from Stanford University.

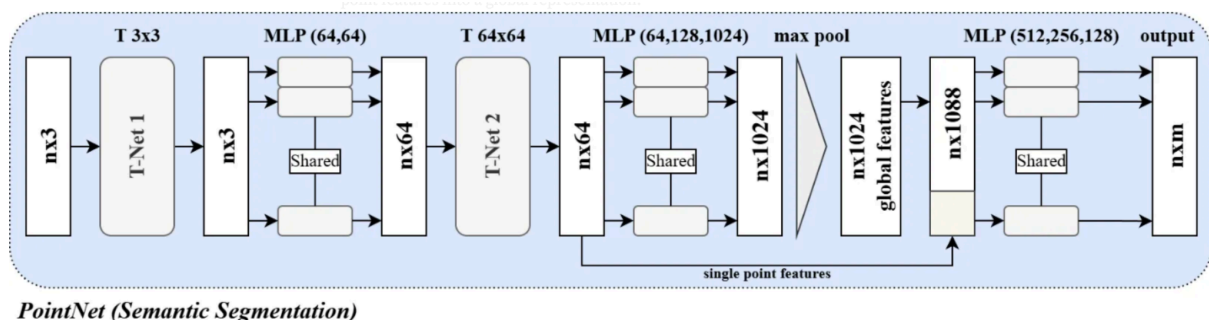


PointNet (the v1 model) either transforms features of *individual points* independently or process global features of the *entire point set*. However, in many cases there are

well defined distance metrics such as Euclidean distance for 3D point clouds collected by 3D sensors or geodesic distance for manifolds like isometric shape surfaces. In PointNet++ we want to respect *spatial localities* of those point sets. PointNet++ learns hierarchical features with increasing scales of contexts, just like that in convolutional neural networks. Besides, we also observe one challenge that is not present in convents (with images) -- non-uniform densities in natural point clouds. To deal with those non-uniform densities, we further propose special layers that are able to intelligently aggregate information from different scales.

We could use PointNet++ for **Shape Classification**, classify ModelNet40 shapes (using point clouds with XYZ coordinates). It can be used for **Object Part Segmentation** as well.

PointNet:



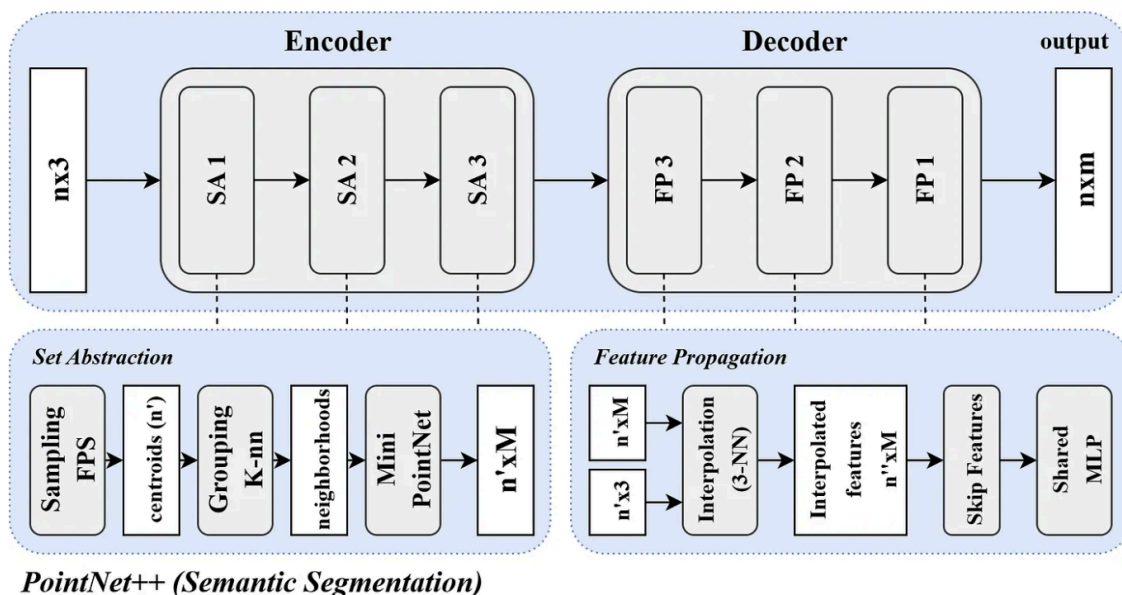
To achieve permutation invariance, PointNet uses a simple and powerful strategy:

- **Shared Multi-Layer Perceptrons (MLPs):** Each point in the cloud is processed independently and identically through a series of shared MLPs. This ensures that the network learns to extract the same type of features for every point, regardless of its position in the input order.
- **Symmetric Function:** A symmetric function, specifically max pooling, is applied across the feature dimension of all points. This function aggregates the learned features from all points into a single global feature vector that describes the entire shape. This aggregation is inherently invariant to the order of the points.
- **Transformation Networks (T-Nets):** To ensure invariance to geometric transformations, PointNet incorporates small neural networks called T-Nets. These networks predict an affine transformation matrix that is applied to the input points and their features, canonicalizing the data before feature extraction.

The key insight of PointNet is to learn spatial encoding for each point individually and then aggregate all individual point features into a global representation.

PointNet++

While PointNet was revolutionary, it did not explicitly capture the local geometric structure induced by the metric space of the points. It treated each point independently before global aggregation, missing fine-grained patterns in local regions. PointNet++ was proposed to address this limitation by introducing a hierarchical feature learning approach that captures patterns at multiple spatial scales.



Block diagram of the PointNet architecture

Encoder

The architecture of PointNet++ is built upon a series of Set Abstraction (SA) modules, which progressively abstract larger regions of the point cloud into high-dimensional feature vectors. Each SA module is composed of three key stages:

- **Sampling Layer:** A subset of points is selected from the input to define the centroids of local regions. The algorithm used is *Farthest Point Sampling (FPS)*, which ensures that the centroids are well-distributed throughout the point cloud, providing a better spatial coverage than random sampling.
- **Grouping Layer:** For each centroid, a local neighborhood is defined by collecting nearby points within a certain radius or using k-nearest

neighbors. These points form a small cluster that captures local geometric information.

- **PointNet Layer:** A mini-PointNet network is applied to each local group to extract a high-level feature vector summarizing the geometric structure of that neighborhood.

By stacking several SA modules, PointNet++ builds an encoding hierarchy: at each level, the number of points decreases while the complexity of the features increases. This process is analogous to convolution and pooling layers in 2D CNNs, which reduce spatial resolution while enriching the representation.

Decoder

For dense prediction tasks such as semantic segmentation, the model must recover per-point predictions for the original resolution. PointNet++ handles this through a **hierarchical feature propagation decoder**, using interpolation and skip connections to transfer and refine features level by level.

- **Interpolation:** During decoding, features from higher-level representations (with fewer points and more global context) are transferred back to denser levels via interpolation. The interpolation is weighted inversely by distance to nearby points.
- **Skip Connections:** Interpolated features are combined with those preserved from the encoder at the same level. This fusion helps recover fine local details that might have been lost during subsampling, while maintaining access to global context.
- **Shared MLP:** After merging, the resulting features are refined through a small shared MLP to update the point-wise representations. This is repeated until all original points receive a final feature vector.

This hierarchical encoder-decoder structure enables PointNet++ to learn robust and detailed representations at multiple scales, achieving superior performance in complex 3D scene understanding tasks.

In our case for car wash we could potentially use PointNet for detection of wheels, mirrors, roof and body of the car.

Advantages of PointNet:

- Made for point clouds
- High accuracy
- Industry standard
- Works directly on XYZ data

Limitations:

- Requires training data
- Training takes weeks
- Requires GPU
- Integration complexity

To integrate this approach to our case, it would take more than 6 weeks since we don't have sufficient amount of data to work with properly.